Operations Research Final Project
Weekly Meal Procurement and Menu Allocation Optimization for a Catering Company

Group 5
b13705022 Chinghua Lu
b13705031 Qianyun Wu
b11702105 Chengsi Tu
t14h06318 Abhirup Sain
t14h06310 Staniya Thomas

1 Introduction

In this project, we study a catering service scenario that supports the industrial workforce of a large-scale manufacturing factory operating continuously, where the catering system must manage resources effectively so that a large number of workers can conveniently access their meals during different shifts. Typically, meal planning is conducted over a fixed planning horizon (e.g., one month), with detailed specifications of dishes and required quantities for each week, and each dish follows a fixed recipe in which the ingredients are predetermined and cannot be substituted.

At the beginning of the planning horizon, all weekly orders are assumed to be known in advance, specifying the number of meals and their compositions. This structured and repetitive demand makes the system suitable for systematic planning and optimization.

To fulfill demand, the catering service must determine the quantity of ingredients to procure. Supplier pricing is rarely linear; in addition to regular unit prices, vendors frequently implement all-units discount structures. Under this model, surpassing a specific volume threshold triggers a retroactive price reduction that applies to the entire order quantity, rather than just the marginal units. This creates a non-linear cost function where the total expenditure may actually decrease at the moment a threshold is met.

Unused ingredients can be stored and carried over to future weeks, assuming sufficient storage capacity. However, since ingredients are perishable and have limited shelf lives, inventory decisions must account for freshness, and expired ingredients must be discarded thereby triggering an additional marginal disposal fee to account for the physical handling, and removal costs of the waste.

This creates a key trade-off for the decision maker: purchasing in larger quantities can reduce unit costs through discounts, but also leads to higher inventory holding costs and increases the risk of holding excess inventory that may expire and result in waste. Balancing these competing factors is central to effective procurement and inventory planning.

In practice, procurement and menu planning are often handled manually, which can be time-consuming and may lead to suboptimal decisions when balancing purchasing costs, inventory holding, and food waste. Therefore, this project aims to develop an optimization model to support decision-making in ingredient procurement and inventory management under fixed recipes, with the objective of minimizing total cost, including purchasing, holding, and waste costs.

2 Mathematical Model

Sets

- T: set of weeks, $T = \{1, \ldots, 4\}$
- O_t: set of orders in week t
- D: set of dishes
- I: set of ingredients

Parameters

- $d_{o,d,t}$: servings of dish d required by order o in week t
- $a_{d,i}$: units of ingredient i required per serving of dish d
- c_i: regular unit purchase cost of ingredient i
- $\bar{c}_i$: discounted unit purchase cost of ingredient i
- m_i: minimum purchase quantity required to activate the discount for ingredient i in a week
- M_i: an upper bound on the discounted purchase quantity of ingredient i
- ε: a small positive constant used to enforce the discount threshold
- h_i: holding cost per unit of ingredient i per week
- w_i: waste cost per unit of ingredient i
- L_i: maximum number of weeks that ingredient i can be stored before it expires

  The upper bound M_i can be set as the maximum cumulative demand of ingredient i over any L_i consecutive weeks, i.e.,

$$M_i = \max_{t \in T} \left(\sum_{\tau=t}^{\min(t+L_i-1,|T|)} \sum_{o \in O_\tau} \sum_{d \in D} a_{d,i} d_{o,d,\tau} \right).$$

Decision Variables

- $q_{i,t} \geq 0$: total purchase quantity of ingredient i in week t
- $q_{i,t}^{\text{reg}} \geq 0$: quantity of ingredient i purchased at the regular unit price in week t
- $q_{i,t}^{\text{disc}} \geq 0$: quantity of ingredient i purchased at the discounted unit price in week t
- $z_{i,t} \in \{0, 1\}$: equals 1 if the discount of ingredient i is activated in week t
- $use_{i,t}^{(a)} \geq 0$: amount of ingredient i of age a used in week t
- $inv_{i,t}^{(a)} \geq 0$: inventory of ingredient i with age a at the end of week t
- $waste_{i,t} \geq 0$: expired quantity of ingredient i in week t

Objective Function

$$\min \sum_{t\in T}\sum_{i\in I}\left(c_i q_{i,t}^{\text{reg}} + \bar{c}_i q_{i,t}^{\text{disc}}\right) + \sum_{t\in T}\sum_{i\in I}\sum_{a=1}^{L_i} h_i\, inv_{i,t}^{(a)} + \sum_{t\in T}\sum_{i\in I} w_i\, waste_{i,t}$$

Constraints

(1) Ingredient Demand from Dishes Ingredient usage must be at least sufficient to satisfy the recipe-derived demand.

$$\sum_{o\in O_t}\sum_{d\in D} a_{d,i}\, d_{o,d,t} \le \sum_{a=1}^{L_i} use_{i,t}^{(a)} \quad \forall i \in I,\ t \in T$$

(2) Purchase Decomposition

$$q_{i,t} = q_{i,t}^{\text{reg}} + q_{i,t}^{\text{disc}} \quad \forall i \in I,\ t \in T$$

(3) Discount Activation

$$\begin{aligned} q_{i,t}^{\text{reg}} &\le M_i(1 - z_{i,t}) \\ q_{i,t}^{\text{disc}} &\ge m_i z_{i,t} \\ q_{i,t}^{\text{disc}} &\le M_i z_{i,t} \\ q_{i,t} &\le (m_i - \varepsilon)(1 - z_{i,t}) + M_i z_{i,t} \end{aligned} \qquad \forall i \in I,\ t \in T$$

(4) Inventory Balance We define $inv_{i,0}^{(a)} = 0$ for all $i \in I,\ a = 1, \ldots, L_i$ and $inv_{i,t}^{(0)} q_{i,t}$ for all $i \in I,\ t \in T$. The inventory balance across all age levels is then written as a single constraint:

$$inv_{i,t}^{(a)} = inv_{i,t-1}^{(a-1)} - use_{i,t}^{(a)} \quad \forall i \in I,\ t \in T,\ a = 1, \ldots, L_i$$

(5) Inventory Availability

$$use_{i,t}^{(a)} \le inv_{i,t-1}^{(a-1)} \quad \forall i \in I,\ t \in T,\ a = 1, \ldots, L_i$$

(6) Waste Definition

$$waste_{i,t} = inv_{i,t-1}^{(L_i)} - use_{i,t}^{(L_i)} \quad \forall i \in I,\ t \in T$$

(7) Inventory Non-negativity

$$inv_{i,t}^{(a)} \ge 0, \quad use_{i,t}^{(a)} \ge 0, \quad waste_{i,t} \ge 0 \quad \forall i, t, a$$

Notes

- **Upper Bound M_i:** The upper bound M_i is defined as the maximum cumulative demand over any L_i consecutive weeks. This bound is tight because purchasing more than this quantity would necessarily lead to expiration due to shelf-life constraints. Since holding and waste costs are non-negative, such decisions are suboptimal.
- **Discount Structure:** We assume an all-units discount structure. If the total purchase quantity $q_{i,t}$ meets or exceeds the threshold m_i, the entire order is billed at the discounted unit cost $\bar{c}_i$; otherwise, the full quantity is billed at the regular unit cost c_i. This is modeled by decomposing $q_{i,t}$ into a regular portion $q_{i,t}^{\text{reg}}$ and a discounted portion $q_{i,t}^{\text{disc}}$, which are made mutually exclusive by the discount activation constraints.
- **Demand Satisfaction:** Ingredient usage must be at least sufficient to satisfy recipe-derived demand.
- **Initial Inventory:** Initial inventory is assumed to be zero for all ingredients and all age levels.

3 Data Collection and Generation

To evaluate the optimization model and implemented solution methods, we construct a set of scenario-based generated instances for the catering procurement problem. The instances are designed to represent a realistic catering setting in which weekly dish demand, fixed recipes, ingredient costs, discount thresholds, shelf lives, holding costs, and waste costs are specified before solving the model. The generated instances are used to test the model and algorithms under different demand patterns, discount conditions, perishability settings, and cost structures.

The data preparation process consists of three main steps. First, dish-level and ingredient-level parameters are organized in spreadsheet format for checking and validation. Second, ingredient demand is derived from dish demand through the recipe matrix. Third, the validated data are stored in a centralized Python instance file so that the same input data can be used by the original greedy heuristic, the improved greedy heuristic, and the Gurobi benchmark model.

3.1 Instance Data Structure

Each instance contains two levels of input data: dish-level data and ingredient-level data.

The dish-level data include weekly dish demand and the recipe matrix. Weekly dish demand specifies the number of servings required for each dish in each week of the planning horizon. The recipe matrix specifies the number of units of each ingredient required to produce one serving of each dish. Since recipes are fixed and ingredient substitution is not allowed, the weekly ingredient demand can be derived directly from dish demand.

The ingredient-level data include regular unit cost, discounted unit cost, discount threshold, shelf life, holding cost, waste cost, and the upper bound M_i used in the discount activation constraints. These parameters determine the purchasing cost, inventory holding cost, waste cost, and the feasibility of discounted purchasing.

In the spreadsheet, these data are checked before being transferred into the Python implementation. The spreadsheet is used to verify the consistency of recipe-based ingredient demand,

recommended M_i values, and discount thresholds. After validation, the same instance data are used by all solution methods to ensure a consistent basis for comparison.

3.2 Dish Demand and Recipe Matrix

The planning horizon consists of four weekly periods. Weekly demand for each dish is assumed to be known in advance, which is reasonable for a catering service where meal requirements are planned before procurement decisions are made.

For Instances 1 to 4, we use a base setting with three dishes and four ingredients. The dishes are $D1$ Tomato-Egg, $D2$ Spinach, and $D3$ Chicken. The corresponding ingredients are Chicken Leg, Tomato, Egg, and Spinach. The recipe structure is fixed as follows:

$$\begin{aligned} D1 \text{ (Tomato-Egg)} &: 1 \text{ unit of Tomato } + 1 \text{ unit of Egg}, \\ D2 \text{ (Spinach)} &: 2 \text{ units of Spinach}, \\ D3 \text{ (Chicken)} &: 1 \text{ unit of Chicken Leg}. \end{aligned}$$

Instance 5 extends the setting to six dishes and six ingredients. The six dishes are Salad, Stir-fry, Roast, Soup, Sandwich, and Bowl. The corresponding ingredients are Chicken, Beef, Tomato, Spinach, Egg, and Rice. This larger instance increases the number of dish-ingredient relationships and is used to test the scalability of the solution methods.

3.3 Ingredient Demand Generation

Ingredient demand is generated from weekly dish demand and the recipe matrix. Let $d_{d,t}$ denote the demand for dish d in week t, and let $a_{d,i}$ denote the amount of ingredient i required for one serving of dish d. The weekly demand for ingredient i in week t, denoted by $D_{i,t}$, is calculated as

$$D_{i,t} = \sum_{d \in D} a_{d,i} d_{d,t}.$$

This transformation ensures that ingredient demand is consistent with the fixed recipe assumption. It also makes the instances easier to revise, since changes in dish demand or recipe coefficients automatically lead to updated ingredient demand.

In the Python implementation, this conversion is performed before solving each instance. The algorithms therefore receive the derived ingredient demand, ingredient parameters, and planning horizon as inputs, rather than handling dish-level demand directly.

3.4 Upper Bound Calculation for Discounted Purchases

For each ingredient, the discount activation constraints require an upper bound M_i on the discounted purchase quantity. Instead of using an arbitrarily large Big-M value, M_i is calculated based on the shelf life and generated demand pattern of each ingredient.

For ingredient i, the recommended upper bound is calculated as the maximum cumulative demand over any L_i-week window, where L_i is the shelf life of ingredient i:

$$M_i = \max_{t \in T} \left(\sum_{\tau=t}^{\min(t+L_i-1,|T|)} D_{i,\tau} \right).$$

This value represents the largest amount of ingredient i that can reasonably be consumed within its shelf-life window. For example, if an ingredient has a shelf life of two weeks, its M_i value is the maximum total demand over any two consecutive weeks. If the shelf life is one week, M_i is the maximum weekly demand over the planning horizon.

Using this bound keeps the formulation tighter and avoids unnecessarily large Big-M values.

3.5 Discount Threshold Validation

For each generated instance, we check whether the discount threshold m_i is structurally consistent with the upper bound M_i. If $m_i \leq M_i$, discounted purchasing can potentially be activated under the generated demand and shelf-life structure. If $m_i > M_i$, the discount is not reachable in that instance.

This validation step does not determine the optimal purchasing decision. It only checks whether the generated parameters are structurally consistent. The actual discount activation decisions are determined by the optimization model or heuristic according to purchasing cost, holding cost, and waste cost.

3.6 Generated Instances

We construct five main generated instances. Instances 1 to 4 are based on the three-dish and four-ingredient setting, while Instance 5 extends the problem to six dishes and six ingredients.

Instance 1 serves as the base case. It includes moderate demand variation across the four-week planning horizon and is used to test the main procurement and inventory mechanisms.

Instance 2 uses lower weekly demand and higher discount thresholds. Under this structure, the discount thresholds are not reachable within the generated demand and shelf-life windows, so the instance represents a no-discount case.

Instance 3 has a more uneven demand pattern, with demand spikes in selected weeks. This instance is used to test cross-week inventory carryover and the timing of purchasing decisions.

Instance 4 uses a flatter demand pattern and serves as the base instance for sensitivity analysis. Starting from this instance, selected parameters of Chicken Leg are varied to examine how changes in cost and discount conditions affect the solution.

Instance 5 extends the setting to six dishes and six ingredients. It is used to examine whether the solution methods remain effective when the number of dishes, ingredients, and recipe interactions increases.

3.7 Sensitivity Scenarios

Sensitivity analysis is conducted based on Instance 4. We modify one selected parameter at a time for Chicken Leg while keeping the rest of the instance structure unchanged. The tested parameters include waste cost, holding cost, and discount threshold.

The waste cost scenarios examine the effect of different disposal penalties. The holding cost

scenarios examine the effect of different inventory storage costs. The discount threshold scenarios examine how the difficulty of activating discounts affects procurement decisions. These scenarios allow us to evaluate whether the solution methods respond consistently to changes in cost structure and discount conditions.

3.8 Solver Input Preparation

After the generated data are checked and validated, they are organized into a solver-readable format. The final input structure includes weekly dish demand, recipe coefficients, ingredient parameters, and derived ingredient demand. This input structure is used consistently by the original greedy heuristic, the improved greedy heuristic, and the Gurobi benchmark model.

Using a shared input structure ensures that all solution methods are evaluated on the same instances. Therefore, differences in total cost, purchasing quantities, discount activation decisions, inventory carryover, waste quantities, and computation time can be attributed to the solution methods rather than inconsistencies in the input data.